PeerConf: Self-Calibrating Early Stopping for Efficient Parallel LLM Reasoning

Mofe Atanda **Yani Baguenane** **Tyler Yi**
Deniz Filiz **Mohammed Mahfoud** **Anusha Mujumdar**
Independent

Abstract

Sampling multiple reasoning traces in parallel significantly improves Large Language Model (LLM) accuracy on complex mathematical problem-solving, yet standard self-consistency runs every trace to completion regardless of quality or early resolution. While confidence-based filtering can mitigate this, current methods rely on a dedicated warm-up phase that wastes significant compute before any filtering begins. We introduce PeerConf (Peer Think with Confidence), a warm-up-free, self-calibrating confidence filter tailored for efficient test-time scaling in mathematical reasoning. PeerConf estimates percentile thresholds dynamically from live-finishing traces within the run itself, allowing filtering and consensus checks to begin immediately at the first finisher. Furthermore, PeerConf introduces reasoning-closure completion probes that test active traces; a trace that clears a confidence threshold and successfully closes its own mathematical reasoning commits early, casts its answer as a vote, and frees its compute seat. Evaluated on challenging competition mathematics benchmarks, including MATH500, AIME25, and HMMT25 using DeepSeek-R1-8B and GPT-OSS-20B, PeerConf matches or exceeds self-consistency accuracy while reducing token consumption by 53.1% to 79.8% relative to self-consistency at 32 traces, outperforming existing warm-up methods in token reduction. By removing the need for separate calibration phases, PeerConf makes extensive test-time scaling practical for resource-constrained mathematical reasoning applications. Code and benchmarks are available at https://github.com/mofe-stack/PeerConf.

1 Introduction

Chain-of-thought prompting improves reasoning by having a model work through a problem step by step [Wei et al., 2022]. Sampling many such traces and voting on the answer is reliable but expensive [Wang et al., 2023]: self-consistency runs every trace to completion, so a low-quality trace costs exactly as much as one that solves the problem. Because parallel sampling still outperforms a single longer chain at equal budget [Ghosal et al., 2025], the question is not whether to sample in parallel but how to stop paying for traces that do not contribute.

DeepConf [Fu et al., 2025] filters traces during generation: it tracks a sliding-window average of token confidence, terminates any trace falling below a threshold, and answers by confidence-weighted majority vote. A per-problem warm-up estimates that threshold. A fixed number of traces run to completion, and DeepConf places the threshold at a percentile of their confidence minima before monitoring begins. The threshold then never moves, so it cannot respond to how the remaining traces perform, however poorly it fits. In our AIME25 replication with 16 warm-up traces and a budget of

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: The 6th Workshop on Mathematical Reasoning and AI.

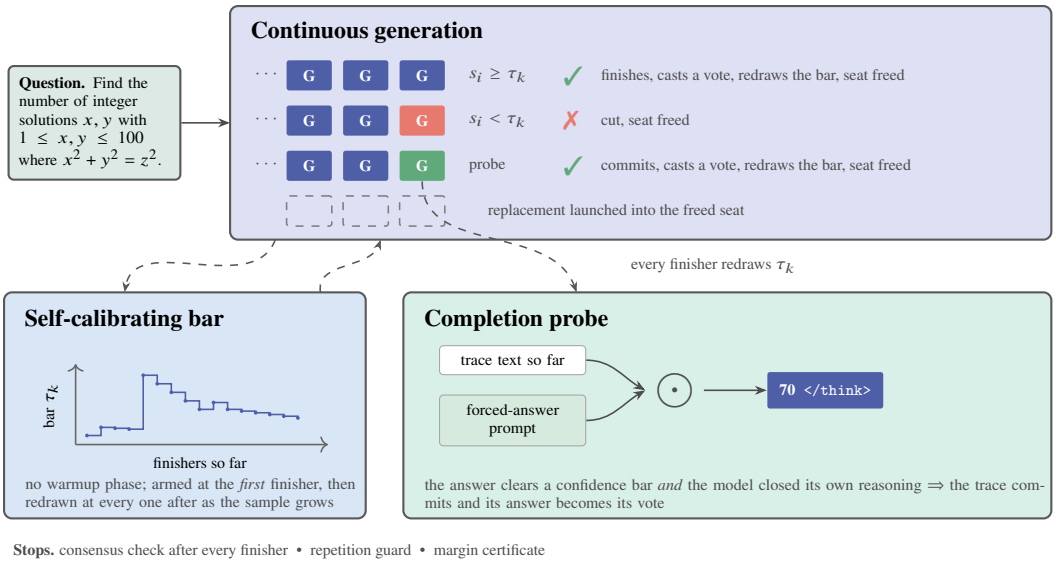


Figure 1: PeerConf during online generation. See Algorithm 1 for the full method.

32, the run generates 95.8% of all tokens before it can cut anything, and on 27 of 30 questions the monitored wave never launches because the warm-up vote already satisfies the consensus condition. The cost lies instead in the tokens a trace keeps generating after it has already reached its answer.

We introduce PeerConf (Figure 1), a warm-up-free, self-calibrating confidence filter that estimates the threshold from the traces that have already finished. Self-calibrating here means the threshold is estimated from the run's own finishers, not calibration in the reliability sense. The threshold arms at the first finisher rather than after a complete warm-up and is recomputed at every finisher thereafter, so both filtering and consensus checks [Aggarwal et al., 2023] begin immediately; DeepConf can end early too, but only once its warm-up completes. Our opening wave is exempt from cutting but is not a warm-up since filtering and consensus begin at its first finisher and not after it completes. A second mechanism addresses traces that have reached their answer but keep generating. Prior work shows that forcing a model to state its answer and stopping once it is confident cuts tokens on a single chain [Hosseini et al., 2026]; we adapt this to the parallel setting and additionally require the probe to close the model's own reasoning, so only traces that have finished reasoning commit. A repetition guard and a margin certificate end generation early; MARS [Chen et al., 2026] relaxes the latter's worst-case bound with a learned per-question estimate of which traces are likely to switch.

Contributions. We estimate the confidence threshold from the run's own finishers rather than a warm-up phase or offline training, so filtering and consensus begin at the first finisher; we adapt forced-answer probing to the parallel setting with a reasoning-closure condition; we measure what committing costs by continuing committed traces to completion; we evaluate against self-consistency and DeepConf with one model on all three benchmarks, and against DeepConf only with a second model on AIME25.

2 Method

2.1 Generation and Calibration

Confidence. We adopt the group confidence of Fu et al. [2025] unchanged: token confidence C_i is the negative mean log-probability of the model's top-k candidates at position i, and a trace's score s_i is the mean of C over a sliding window of the W most recent tokens, defined only on complete windows (Appendix B). We summarize a trace by its lifetime minimum $m = \min_{i \ge W} s_i$, which is non-increasing, so a trace whose minimum has fallen below a threshold cannot later rise above it. We use $k = 20$ throughout, with $W = 2048$ on AIME25 and HMMT25 and $W = 256$ on MATH500.

Generation and budget. A run answers one question with at most N traces and K replacements generating concurrently after an opening wave of K_0; every departure frees a slot until N traces have launched. As in self-consistency [Wang et al., 2023], all traces are sampled independently from the

same prompt. Opening-wave traces are exempt from termination and run until they answer or reach the cap, but are still filtered at the vote by the same threshold, as Fu et al. [2025] do with warm-up traces. In our experiments $K_0 = K = 16$ and $N = 32$.

In-run calibration. We call a trace a *finisher* when it finishes with an answer, reached on its own or committed to through a completion probe. Every finisher contributes its minimum m to the calibration set, so replacements calibrate alongside the opening wave and the set grows until the run ends.

Arming and recomputation. We estimate the threshold with the same percentile rule of Fu et al. [2025] applied to the minima from the paths collected so far:

$$\tau_k \;=\; \text{percentile}\big(M_k,\; 100 - \eta\big), \tag{1}$$

where M_k holds the minima of the k finishers and η is the retention ratio. At $\eta = 10$ the threshold keeps the top 10% of the finishers' minima, which is the setting Fu et al. [2025] call DeepConf-low. At $\eta = 90$ it keeps the top 90 percent, DeepConf-high. We use the same settings for our threshold which we call PeerConf-low and PeerConf-high, and in our experiments we ran PeerConf-low and DeepConf-low; we did not evaluate the high variants due to computational constraints. DeepConf estimates the percentile once over a fixed warm-up set while ours arms on the first finish and is recomputed on every finish after it.

2.2 Filtering, Completion, and Stopping

Cutting & Loop Termination. A replacement ends the moment one of its windows falls below τ_k; the opening wave is exempt from termination. Following Fu et al. [2025], each newly completed window is compared against the current threshold, and earlier windows are not re-checked when the threshold rises. A trace repeating itself verbatim is also terminated, casts no vote, and does not enter the calibration set (parameters in Appendix E).

Completion probes. We append a short forced-answer trigger to the trace so far and generate at most 20 greedy tokens (Appendix C).

Stopping and aggregation. After every departure the votes are recomputed, so a question can end the moment its answer settles. Following Aggarwal et al. [2023] and the adaptive sampling rule of Fu et al. [2025], we stop once the leading answer holds $\tau_c = 0.95$ of the confidence-weighted vote among at least three finishers. During the run, opening-wave finishers are filtered by the threshold only after the whole opening wave has departed; the final vote always applies the filter. We report $\hat{a}$, the answer holding the most confidence-weighted vote. A margin certificate additionally stops the run once the leader's margin exceeds every vote the unfinished and unlaunched traces could contribute, an exact head count rather than a probabilistic bound. The count is neither weighted nor filtered, so it certifies the plurality among finished traces and is not a guarantee about $\hat{a}$ (Appendix E).

3 Experiments

Models and benchmarks. DeepSeek-R1-0528-Qwen3-8B [DeepSeek-AI, 2025] is our primary model and carries all ablations; GPT-OSS-20B [OpenAI, 2025] is a second model family. Sampling follows each model's published configuration. We evaluate on three competition mathematics datasets: AIME25 [Zhang and Team, 2025], HMMT25 [HMMT, 2025] and MATH500 [Lightman et al., 2023]. HMMT25 answers are frequently symbolic rather than integer-valued, making answer normalization substantive; MATH500 problems are shorter-horizon than either competition set (Appendix E).

Baselines and setup. Self-consistency samples the full 32-trace budget to completion with no filtering and answers by plain majority vote. DeepConf-low uses the same budget, estimating its threshold from 16 warm-up traces at the top 10% of their minimum window confidences and freezing it thereafter. We evaluate PeerConf-low with completion probing, the repetition guard, and the margin certificate enabled. All methods share the same model, budget, sampling settings, serving engine, confidence computation, answer extractor, and grader, and DeepConf and PeerConf answer by the same confidence-weighted vote (Appendix E). PeerConf differs from DeepConf in its calibration scheme, completion probes, repetition guard, and margin certificate, and we do not isolate the contribution of each.

Table 1: Performance comparison across datasets, windows, and methods. PeerConf-low achieves strong accuracy while significantly reducing token usage.

Model	Dataset	Window	Method	Acc	Total tokens*
DeepSeek-R1-8B	MATH500	256	SC@32	89.8%	122.38M
			DeepConf	91.0%	63.84M (−47.8%)
			PeerConf	91.0%	24.73M (−79.8%)
	AIME25	2048	SC@32	83.3%	25.60M
			DeepConf	83.3%	13.38M (−47.7%)
			PeerConf	83.3%	10.06M (−60.7%)
	HMMT25	2048	SC@32	66.7%	28.34M
			DeepConf	76.7%	17.31M (−38.9%)
			PeerConf	73.3%	13.30M (−53.1%)
GPT-OSS-20B	AIME25	2048	DeepConf	83.3%	18.26M
			PeerConf	86.7%	9.84M (−46.1%)

Note: Parenthesised percentages are the token reduction against self-consistency at 32 traces where we ran it, and against DeepConf on the same questions otherwise. PeerConf's totals include the tokens its completion probes generate.

3.1 Main Results

As summarized in Table 1, PeerConf-low matches or exceeds self-consistency on every benchmark while generating 53.1–79.8% fewer tokens: 79.8% on MATH500, 60.7% on AIME25, and 53.1% on HMMT25, against DeepConf's 47.8%, 47.7%, and 38.9%. Savings are largest on MATH500, where simpler problems let traces commit early via completion probes while the run reaches consensus without waiting on a warm-up. On AIME25 with GPT-OSS-20B, PeerConf saves 46.1% relative to DeepConf while exceeding its accuracy by 3.3 points. HMMT25 is the one benchmark where PeerConf trails DeepConf, by 3.3 points, though both exceed self-consistency: the benchmark's greater difficulty yields less agreement, so consensus arrives later and on fewer questions, and traces explore throughout their length, leaving little post-solution redundancy. Notably, both 3.3-point differences correspond to a single question on these 30-question benchmarks.

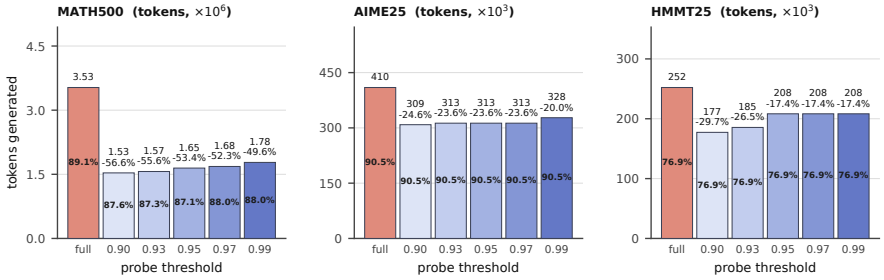


Figure 2: Ghost run on DeepSeek-R1-8B: accuracy and token cost for questions committing at each threshold. Accuracy inside bars, tokens above. Filtered to questions committing at one or more thresholds (442/500, 21/30, 13/30). Each question has one trace, while Table 1 votes over 32 traces.

In Figure 2 we isolate the cost of committing early by running one trace per question, letting every committed trace run to full completion, and recording where each probe would have committed. Filtering to questions that commit at one or more thresholds leaves 442 of 500 questions on MATH500, 21 of 30 on AIME25, and 13 of 30 on HMMT25. On AIME25 and HMMT25 accuracy is identical to full completion at every threshold from 0.90 to 0.99 while tokens fall by 20–25% and 17–30% respectively; MATH500 costs 1.1–2.0 percentage points of accuracy for savings of 49.6–56.6%. Appendix F plots where each MATH500 trace ended against where a probe would have stopped it.

Ablation: Reasoning Closure. In Figure 3 we ablate the reasoning-closure requirement on MATH500. Closure is what stabilizes accuracy: with both conditions required, accuracy never falls below 85.5% and rises as the threshold tightens, from 85.9% at 0.90 to 86.9% at 0.999, against 87.3% for full completion. Confidence alone begins at 76.5%, more than ten percentage points

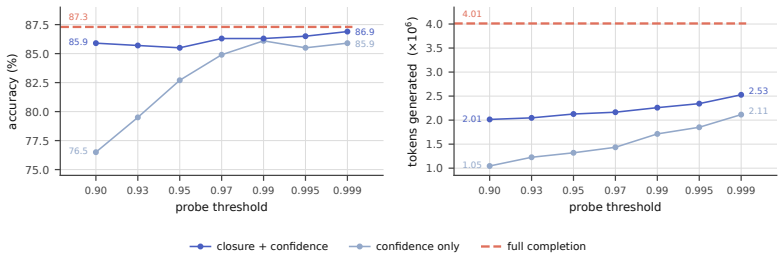


Figure 3: Ablation study on the reasoning-closure requirement on MATH-500. A trace commits only when its probe clears the confidence bar *and* closes the model's own reasoning. Filtered to questions that commit at one or more thresholds under either requirement (498/500 for MATH500). Confidence alone peaks at 0.99 and does not improve further, while adding closure keeps improving to within 0.4 percentage points of full completion.

lower, and recovers only as the threshold tightens, peaking at 86.1% without improving further, while closure plus confidence outperforms its accuracy at each threshold. A confident answer alone cannot tell whether the model has finished its reasoning. Therefore a confidence-only threshold cannot remove traces that commit to a wrong answer while very confident mid-thought, while closure does. Closure still removes 37–50% of generation, against 47–74% for confidence alone, so requiring the model to finish its own reasoning buys a stable, improving accuracy curve at a modest fraction of the savings.

4 Discussion and Limitations

Static hyperparameters. PeerConf's probe period, sliding window, and trace-length cap are fixed constants applied uniformly across questions rather than adapted to difficulty, and the rigidity extends to compute and path allocation. We observe substantial variance in difficulty: some questions require only a 32K cap, others up to 64K. A difficulty-aware scheme that scales compute and path budgets per question is the natural next step.

Accuracy against confidence. Our sliding window assumes a more confident path is a more accurate one, but on some questions across several benchmarks we observe the inverse. Paths that reach an answer quickly without double-checking carry inflated confidence, while paths that reason thoroughly and verify accumulate lower confidence across their length, leaving them vulnerable to premature termination or exclusion from the vote.

Scope and baseline implementation. Our evaluation is limited to competition-mathematics benchmarks at the 8B–20B scale, with every run capped at 32 traces and 64,000 tokens per trace. All results are single runs per question without error bars; on the 30-question benchmarks a 3.3-point difference is one question. Removing the warm-up eliminates a fixed overhead whose relative contribution should shrink as the budget grows, while completion probes cut tokens within every trace; we do not test this, as every run uses $N = 32$. The end-of-thinking marker is a stopping heuristic: it does not verify that the reasoning is complete or correct, and it requires a model with an explicit thinking marker. We report generated tokens only, so the input tokens each probe re-processes are not counted and we do not measure wall-clock or GPU time. Finally, because DeepConf's adaptive sampling was absent from its official release, we re-implemented it from the paper and verified it reproduces the reported behavior; minor discrepancies may remain.

5 Conclusion

PeerConf replaces DeepConf's warm-up-based threshold with a live-calibrating bar and adds completion probes that stop paths which have already reached their answer, substantially reducing token consumption while retaining competitive accuracy. Cutting the tokens that parallel reasoning requires should lower its monetary and energy cost, widening access for practitioners with limited compute, while lowering the cost of deploying reasoning models for any application.

References

Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with llms. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://arxiv.org/abs/2305.11860.

Wenbo Chen, Puheng Li, Mengyang Liu, Weijie Su, and Tianpei Xie. Mars: Margin-adversarial risk-controlled stopping for parallel llm test-time scaling, 2026. URL https://arxiv.org/abs/2606.12935.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Yonghao Zhuang, Yian Ma, Aurick Qiao, Tajana Rosing, Ion Stoica, and Hao Zhang. Efficiently scaling llm reasoning with certaindex, 2024. URL https://arxiv.org/abs/2412.20993.

Yichao Fu, Xuewei Wang, Yuandong Tian, and Jiawei Zhao. Deep think with confidence. *arXiv preprint arXiv:2508.15260*, 2025.

Soumya Suvra Ghosal, Souradip Chakraborty, Avinash Reddy, Yifu Lu, Mengdi Wang, Dinesh Manocha, Furong Huang, Mohammad Ghavamzadeh, and Amrit Singh Bedi. Does thinking more always help? mirage of test-time scaling in reasoning models. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2506.04210.

HMMT. Harvard-MIT mathematics tournament 2025. https://www.hmmt.org, 2025.

Parsa Hosseini, Sumit Nawathe, Mahdi Salmani, Meisam Razaviyayn, and Soheil Feizi. Early stopping for large reasoning models via confidence dynamics, 2026. URL https://arxiv.org/abs/2604.04930.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023. URL https://arxiv.org/abs/2305.20050.

OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

Shangqing Tu, Yaxuan Li, Yushi Bai, Lei Hou, and Juanzi Li. Deepprune: Parallel scaling without inter-trace redundancy, 2025. URL https://arxiv.org/abs/2510.08483.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. arXiv:2203.11171.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2201.11903.

Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models, 2025. URL https://arxiv.org/abs/2504.15895.

Qinglin Zeng, Jing Yang, and Keze Wang. Reflective confidence: Correcting reasoning flaws via online self-correction, 2025. URL https://arxiv.org/abs/2512.18605.

Qinglin Zeng, Jusheng Zhang, Jing Yang, Ningyuan Liu, and Keze Wang. RACC: Regret-aware confidence calibration for consistent masked discrete diffusion decoding. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. URL https://aclanthology.org/2026.findings-acl.1138/.

Yifan Zhang and Math-AI Team. American invitational mathematics examination (aime) 2025. https://huggingface.co/datasets/math-ai/aime25, 2025.

A Background and Related Work

Confidence-filtered parallel reasoning. A single chain of thought [Wei et al., 2022] fails whenever that one path goes wrong, and extending it further adds variance rather than accuracy [Ghosal et al., 2025]. Self-Consistency [Wang et al., 2023] instead samples many chains and votes, at a cost linear in the number of paths, since every trace runs to completion regardless of how badly it is going. Scoring traces after they finish improves answer selection but saves nothing during generation [Lightman et al., 2023]. DeepConf [Fu et al., 2025] moves the check online, averaging token confidence over a sliding window and terminating any trace whose windowed confidence falls below a threshold, reducing tokens by up to 84.7% at a 512-trace budget. Its run proceeds in two stages: a warm-up of 16 traces first runs to completion, the threshold is estimated from their confidence minima, and that same threshold then excludes insufficiently confident warm-up traces from the ballot; consensus among the remainder is then checked. Only if no consensus is reached are the remaining traces generated under the now-fixed threshold, with consensus rechecked as each finishes, which they call adaptive sampling. We adopt its confidence definition, percentile rule, discard decision, and vote filter unchanged, and change when and from what the threshold is formed.

Threshold calibration. Current practice estimates the threshold from a dedicated warm-up: traces are generated first, their confidence minima collected, and the threshold estimated at a percentile of that distribution. This calibration is per problem in both DeepConf [Fu et al., 2025] and Reflective Confidence [Zeng et al., 2025], which spends half of its budget allocating 16 of 32 traces to warm-up, and reappears in MARS [Chen et al., 2026], which calibrates an adversarial bound from warm-up traces while additionally fitting a per-question model of answer switching. These methods share a structural property: no filtering or consensus decision can be made until calibration completes, so every warm-up trace runs to its natural end regardless of whether its answer has already settled, and the threshold is frozen thereafter and cannot respond to evidence accumulating while the problem is solved. A threshold defined relative to a run's own recent behavior has proved effective in masked diffusion decoding [Zeng et al., 2026], in a setting not directly comparable to ours. Some set the threshold with offline training instead, by fitting a judge and fixing one threshold on its scores to prune redundant paths at run time [Tu et al., 2025]. In contrast to all of these methods, we estimate our threshold as a percentile over the confidence minima of traces finishing within the run itself, armed at the first finish and recomputed at every finish thereafter. Filtering and consensus checks therefore begin from the first finisher rather than after the warm-up phase, and calibration comes from the traces already being voted on, so neither a dedicated warm-up nor offline training is required. We retain DeepConf's voting filter over the opening wave, so those traces are candidate rather than guaranteed voters.

Early stopping criteria. Interrupting a trace to elicit its current answer is cheap and informative, and single-trace methods differ mainly in what they do with the elicited answer: Dynasor-CoT [Fu et al., 2024] probes at short fixed intervals and stops once the same answer recurs across consecutive probes; DEER [Yang et al., 2025] induces a trial answer at transition points and stops when the mean token probability of that answer clears a threshold; and CoDE-Stop [Hosseini et al., 2026] monitors the confidence of successive elicited answers and stops once the model has committed, cutting tokens by 25–50%. With one trace and no vote, a confident wrong completion goes unchecked in all three. We adapt this mechanism to the parallel setting, where a committed trace stops generating, casts its probe answer as its vote, and frees its seat for a replacement. Like DEER we score the probe on its answer tokens alone; unlike any of these we accept the result only when the model closes its own reasoning within the probe, so a trace commits because it is genuinely finished rather than because a threshold was crossed mid-thought. MARS [Chen et al., 2026] also probes active traces in parallel, but uses the extracted answers together with a learned per-question model of answer switching to certify when the run as a whole can halt, saving 14–29% on top of DeepConf Online, where they remove DeepConf's adaptive sampling component. Where that work estimates whether a trace would have changed its answer, we measure it directly by continuing committed traces to completion. Additionally, our margin rule is the deterministic worst case of the same certified criterion, where every trace that has not voted yet is assumed to vote against its leader. Because a looping trace emits confident tokens that no threshold can flag, we pair the filter with a text-level repetition guard.

B Confidence Definitions

Following Fu et al. [2025] we score each generated token by the negative mean log-probability of the model's top-k candidates at that position,

$$C_i = -\frac{1}{k}\sum_{j=1}^{k}\log P(x_{i,j}), \tag{2}$$

so a peaked next-token distribution scores a high C_i and a flat one a low C_i. Token-level scores are too variable to judge a trace by, so we evaluate a trace over a sliding window of the W most recent tokens,

$$s_i = \frac{1}{W}\sum_{t=i-W+1}^{i} C_t, \qquad i \geq W, \tag{3}$$

which is the group confidence of Fu et al. [2025], evaluated only on complete windows. We summarize a trace by the lowest window it reaches over its lifetime,

$$m = \min_{i \geq W} s_i, \tag{4}$$

the lowest group confidence of Fu et al. [2025]. The minimum is taken from the first complete window, since s_i is undefined for $i < W$. Fu et al. [2025] likewise report multiple window lengths ($n = 1024$ or 2048).

C Completion Probe

At 9,003 tokens. $c = 0.343$, 20 tokens used, no marker. No completion.
The final answer is `\boxed{12250}. However, I did not complete the calculation....`

At 21,291 tokens. $c = 0.999$, 7 tokens used, ended on the marker. Commits.
The final answer is `\boxed{6300}.`

Figure 4: Two completion probes on one HMMT25 trace and the correct answer 6300. Grey text is generated by the model and everything before it is the probe prompt. While the reasoning is unfinished the probe answers wrongly and therefore runs out its 20-token budget. Once the trace is ready it answers in seven tokens and stops on the end of thinking marker. In order for the trace to commit to the probe's answer it must meet both conditions. Token counts are the trace's length when each probe returned; because the trace continues generating while the probe runs, they lag the nominal Π-token schedule.

A trace commits to the probe's answer and stops if

$$c = \exp\left(\frac{1}{|A|}\sum_{j\in A}\log p_j\right) \geq \gamma, \tag{5}$$

where A is the answer tokens the probe returns and p_j their probabilities, and the probe ended on its own on an end of thinking marker (e.g. `</think>` or `<|end|>`) rather than running out the 20 tokens. We use $\gamma = 0.95$ in our experiments, and the probe period is dependent on the benchmark: 4096 on AIME25 and HMMT25, 512 on MATH500.

D PeerConf Algorithm

Algorithm 1 PeerConf

Inputs: prompt P, budget N, opening wave K_0, replacement seats K, retention ratio η, window W, probe period Π, probe threshold γ, consensus level τ_c
Opening Wave:
Launch $t_0, \ldots, t_{K_0-1}$ on prompt P; calibration set $M \leftarrow \emptyset$; threshold $\tau \leftarrow$ unset
Online Generation:
while any trace is still generating **do**
  while generating trace t, alongside every other live trace **do**
    Generate the next token, update the window score s_i, and update $m_t = \min_{i \geq W} s_i$
    If t is a replacement and its newest window score $s_i < \tau$: stop generating t, no vote
    If the last 120 characters of t appear three or more times in its last 2400: stop generating t, no vote
    Every Π tokens: append the forced-answer trigger and generate at most 20 greedy tokens; **if** the probe ends on the end of a thinking marker with confidence $c \geq \gamma$: commit t to that answer and stop generating t
  end while
  If t finished with an answer: $M \leftarrow M \cup \{m_t\}$ and $\tau \leftarrow \text{Percentile}_{100-\eta}(M)$
  If at least three traces have finished and the leading answer holds τ_c of the vote: stop every trace still generating
  If the leading answer's ballot margin over all finished traces exceeds the number still outstanding: stop every trace still generating
  Seat a replacement while fewer than K replacements are generating and fewer than N traces have launched
end while
return the answer with the highest weighted vote while excluding opening-wave traces with $m_t < \tau$

E Implementation Details

Benchmarks. AIME25 comprises the 30 problems of the 2025 American Invitational Mathematics Examination, with integer answers in $[0, 999]$. HMMT25 comprises the problems of the 2025 Harvard–MIT Mathematics Tournament; its answers are frequently symbolic rather than integer-valued, including $8\sqrt{10}$, $9\sqrt{15}$, $\sqrt{6}$, $\frac{1}{576}$, and negative integers such as -984, which makes answer normalization substantive rather than a formality. MATH500 is a 500-problem subset of MATH whose problems are shorter-horizon: a trace run to completion requires fewer tokens than on either competition benchmark.

Models and sampling. DeepSeek-R1-0528-Qwen3-8B uses temperature 0.6, top-p 0.95, no top-k, 20 retained logprobs, and a 64,000-token cap per trace. GPT-OSS-20B uses temperature 1.0, top-p 1.0, and top-k 40. All traces for a question are sampled independently from the same prompt.

Serving. All runs use a single unmodified vLLM 0.9.2 server per session at bfloat16 precision with a 65,536-token context, on 2x H200's. Confidence is computed client-side from streamed log-probabilities rather than by a server-side logits processor, so the serving configuration is identical across methods and the method under evaluation is the only point of difference. Answer equivalence uses `math_equal` from the Dynasor evaluation suite.

Method settings. $k = 20$ retained logprobs; opening wave $K_0 = 16$ and replacement seats $K = 16$; budget $N = 32$; retention ratio $\eta = 10$ (PeerConf-low); consensus level $\tau_c = 0.95$; probe threshold $\gamma = 0.95$; probe period $\Pi = 4096$ on AIME25 and HMMT25 and 512 on MATH500; window $W = 2048$ on AIME25 and HMMT25 and 256 on MATH500. DeepConf-low uses the same budget with 16 warm-up traces and the same retention ratio and consensus level.

Repetition guard. Every 256 generated tokens we count how many times a trace's last 120 characters appear within its last 2400. If the string appears three or more times the trace is terminated, casts no vote, and is excluded from the calibration set.

Consensus-based early stopping. After every departure the votes are recomputed. Let F be the traces that have finished and still vote; each contributes its minimum, so answer a holds

$$V(a) = \sum_{t \in F,\ a_t = a} m_t, \qquad \hat{a} = \arg\max_a V(a). \tag{6}$$

Fu et al. [2025] use adaptive sampling as their consensus rule, comparing the leading answer's share of the vote against τ_c. We use the same rule, $V(\hat{a})/\sum_a V(a) \geq \tau_c$, while additionally requiring at least three finishers. We set $\tau_c = 0.95$, the value selected in their own ablation.

Margin certificate. The certificate counts one vote for every finished trace and compares the leader's margin against the traces still generating and those not yet launched. If the margin is larger than that number, no arrangement of the remaining answers can overtake the leader, so nothing is gained by generating them and the run ends. The count is exact rather than probabilistic, being a head count over finished traces that are neither weighted nor filtered; it is therefore not a guarantee about $\hat{a}$.

F MATH500 Path Lengths

Ordered by trace length, Figure 5 plots where each MATH500 trace ended against where a probe at 0.97 would have stopped it. The gap totals 1.8M tokens never generated, 46% of the benchmark, and widens with trace length: the longest traces continue for tens of thousands of tokens after their answer has settled.

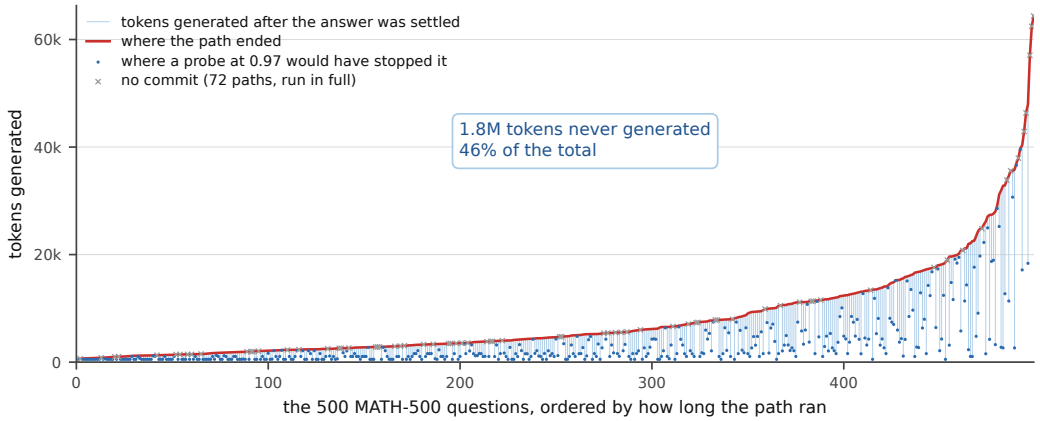


Figure 5: Where each MATH-500 path ended naturally against where a probe at 0.97 would have committed it in tokens generated (DeepSeek-R1-8B)

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: In all benchmarks, PeerConf saves more tokens than SC@32 and DeepConf, as shown in Section 3.1 and Table 1.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Limitations are addressed in Section 4. Limitations cover static hyperparameters, flawed correlation between accuracy and confidence, and the limited scope of the paper.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The papers contributions are empirical, not theoretical.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 2 specifies every component of the method and Algorithm 1 gives the full procedure. Section 3 and Appendix E reports the models, sampling parameters, token caps, benchmarks, per-benchmark window and probe periods, baseline configurations, serving stack, and the answer-equivalence routine. No training is involved, so there are no splits, optimizers, or checkpoints to disclose.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Instructions live inside of the pubic repo at https://github.com/mofe-stack/PeerConf, allowing for faithful reproductions of results.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 2 and Appendix E specifies the models and sampling parameters, the three benchmarks and the question subsets used, the baseline configurations, PeerConf's settings, and the serving and grading stack. Section 3 gives the remaining method hyperparameters, including the window length, probe period, retention ratio, and consensus level.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: PeerConf evaluation against MATH500, AIME25, and HMMT25 evaluates the entire benchmark population, thus we report the exact point estimates for token usage and accuracy across the paper rather than sample error bars. Results are single run given compute constraints, and the HMMT25 and AIME25 Benchmarks are 30 question sets where one flipped answer changes accuracy by 3.3 points, comparable in size to several reported Deltas.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix E reports the serving configuration: a single unmodified vLLM 0.9.2 server per session at bfloat16 precision with a 65,536-token context on 2x H200 GPUs. Per-method token totals for every benchmark are given in Table 1. Detailed instructions for faithful replications of experiments and code are situated in the public repo.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research conforms to the NeurIPS Code of Ethics. It evaluates inference-time procedures on publicly available mathematics benchmarks using publicly released models, and involves no human subjects, personal data, or newly collected data.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Broader impacts are discussed in Section 5. The method reduces the compute and energy required for parallel reasoning, while lowering inference cost also lowers the barrier to deploying reasoning models generally, including any application.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper releases no models, datasets, or other assets carrying a risk of misuse. It describes an inference-time procedure applied to already-released models and publicly available mathematics benchmarks.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All benchmark datasets (MATH500, AIME25, HMMT25) and open source models (Deepseek-R1-8B, GPT-OSS-20B) are properly cited and utilized within their public licenses.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: PeerConf is a new method building off previous advancements in parallel reasoning. We use existing, open-source assets for our models (Deepseek-R1-8B, GPT-OSS-20B) and datasets (MATH500, AIME25, HMMT25).

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no research with human subjects. All evaluation is automatic against published answer keys.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper involves no crowdsourcing and no research with human subjects, so no IRB review or equivalent was required.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: Although LLMs were not utilized to implement the core method, they represent the empirical baseline and subject of evaluation. Comprehensive model specifications and parameters are provided in Section 3 and Appendix E.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.